

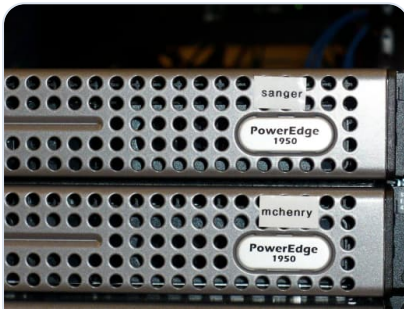
INTRODUCTION AU TEST D'INTRUSION

Guide du TP 4 : trier un rapport de scanner sans sur-réagir

TP 4 · Trier un rapport de scanner



🔧 MATÉRIEL QUE TU CROISES DANS CE LABORATOIRE



Serveur

Fournit fichiers, messagerie ou site web aux autres



Stockage en réseau (NAS)

Dossier partagé et sauvegarde pour toute l'équipe

Photos issues de Wikimedia Commons : Serveur, Florence Devouard, CC BY-SA 4.0 · Stockage en réseau (NAS), DYVER, CC BY-SA 4.0.

💡 À RETENIR

- ◆ **Cadre et limites** : tu analyses uniquement des fichiers fournis et fictifs. Ne lance aucun scan sur un système réel et ne teste rien sans autorisation écrite.
- ◆ **Les exemples de ce guide** utilisent des valeurs inventées et un mini jeu de données local créé dans les blocs. Ils n'utilisent jamais les fichiers du labo.

1 Ce que tu dois décider

Alerte scanner ne veut pas dire constat confirmé. Tu dois d'abord déterminer si le constat est confirmé, faux positif ou non confirmable faute de preuve minimale.

- ◆ **confirme** : la preuve minimale relie clairement le produit, la version ou le comportement observé à l'alerte ;
- ◆ **faux positif** : la version réellement observée n'est pas touchée ou la signature matche un cas déjà corrigé ;
- ◆ **preuve insuffisante** : le scanner donne une piste, mais la preuve minimale ne montre ni version utile, ni comportement exploitable, ni configuration assez précise.

2 Les poids CVSS 3.1 à utiliser

- ◆ AV : $N=0.85$, $A=0.62$, $L=0.55$, $P=0.20$
- ◆ AC : $L=0.77$, $H=0.44$
- ◆ PR si S:U : $N=0.85$, $L=0.62$, $H=0.27$
- ◆ PR si S:C : $N=0.85$, $L=0.68$, $H=0.50$
- ◆ UI : $N=0.85$, $R=0.62$
- ◆ C, I, A : $H=0.56$, $L=0.22$, $N=0.00$

3 Formule CVSS 3.1

- 1 Calcule $ISS = 1 - (1-C) \times (1-I) \times (1-A)$.
- 2 Si S:U, $Impact = 6.42 \times ISS$.
- 3 Si S:C, $Impact = 7.52 \times (ISS - 0.029) - 3.25 \times (ISS - 0.02)^{15}$.
- 4 $Exploitability = 8.22 \times AV \times AC \times PR \times UI$.
- 5 Si $Impact \leq 0$, la note vaut 0.0 .
- 6 Sinon :
 - ◆ S:U : $Roundup(\min(Impact + Exploitability, 10))$
 - ◆ S:C : $Roundup(\min(1.08 \times (Impact + Exploitability), 10))$

La fonction **Roundup** arrondit au dixième supérieur. Si le résultat tombe déjà exactement sur un dixième, il reste tel quel. Sinon, on monte au dixième suivant.

4 Qualification d'une note

- ◆ 0.0 : aucune
- ◆ 0.1 à 3.9 : faible
- ◆ 4.0 à 6.9 : moyenne
- ◆ 7.0 à 8.9 : élevée
- ◆ 9.0 à 10.0 : critique

5 EPSS, KEV et règle de priorisation du dossier

EPSS est une probabilité entre 0 et 1 d'exploitation observée dans les 30 jours, publiée par FIRST. KEV est le catalogue CISA des vulnérabilités exploitées connues ; ce n'est pas une mesure de gravité. Pour ce TP, trie dans cet ordre :

- 1 exclus d'abord les lignes `faux_positif` et `preuve_insuffisante` ;
- 2 parmi les constats confirmés, `kev=yes` passe avant `kev=no` ;
- 3 ensuite compare la **bande EPSS** : `>=0.85` très haute, `0.60-0.849` haute, `0.30-0.599` moyenne, `<0.30` faible ;
- 4 à bande EPSS identique, compare la criticité de l'actif : rang 5 avant 4, puis 3, puis 2 ;
- 5 si le rang d'actif est identique, compare la valeur EPSS brute la plus haute ;
- 6 s'il reste un ex æquo, garde l'identifiant le plus petit.

Cette règle permet à un constat sur un actif métier critique de passer devant un constat presque équivalent sur un site vitrine, même si sa valeur EPSS brute est légèrement plus basse.

6 Visionneuse du site

Dans la visionneuse, ouvre d'abord le CSV du scanner pour repérer les identifiants `Vxx`, puis le fichier de validation pour voir le `statut` et la `decision_vecteur`, puis le CSV EPSS/KEV et enfin la fiche des actifs.

7 Exemple Bash : filtrer les constats confirmés et compter les KEV

```
cat > demo-scanner.csv <<'EOF'
id,produit
V01,Exemple Portal
V02,Exemple Queue
V03,Exemple Cache
EOF
cat > demo-notes.txt <<'EOF'
[V01] statut=confirme preuve=suffisante decision_vecteur=conserve
vecteur_final=CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
[V02] statut=faux_positif preuve=suffisante decision_vecteur=rejete vecteur_final=-
[V03] statut=confirme preuve=suffisante decision_vecteur=ajuste vecteur_final=CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
EOF
cat > demo-epss.csv <<'EOF'
id,epss,kev,fenetre_jours
V01,0.812,yes,30
V02,0.955,yes,30
V03,0.421,no,30
EOF
grep 'statut=confirme' demo-notes.txt | cut -d']' -f1 | tr -d '[' | while read -r ident; do
  awk -F, -v id="$ident" 'NR>1 && $1==id && $3=="yes" { print $1 " " $2 }' demo-epss.csv
done
```

Commande longue, coupée par la mise en page : copiez-la en une seule ligne ou utilisez la version texte indiquée en fin de document.

8 Exemple PowerShell 5.1 : même tri sur des données inventées

```
@(
    'id,epss,kev,fenetre_jours',
    'V01,0.812,yes,30',
    'V02,0.955,yes,30',
    'V03,0.421,no,30'
) | Set-Content -Encoding UTF8 demo-epss.csv
@()
```

```
'[V01] statut=confirme preuve=suffisante decision_vecteur=conserve
vecteur_final=CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N',
'[V02] statut=faux_positif preuve=suffisante decision_vecteur=rejete vecteur_final=-',
'[V03] statut=confirme preuve=suffisante decision_vecteur=ajuste
vecteur_final=CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N'
) | Set-Content -Encoding UTF8 demo-notes.txt
$confirmes = Get-Content -Encoding UTF8 demo-notes.txt | Where-Object { $_ -match 'statut=confirme' } | ForEach-Object {
    if ($_.match -match '^\[(\d{2})\]$') { $Matches[1] }
}
Import-Csv -Encoding UTF8 demo-epss.csv | Where-Object { $_.kev -eq 'yes' -and $confirmes -contains $_.id } | ForEach-Object {
    '{0} {1}' -f $_.id, $_.epss
}
```

Commande longue, coupée par la mise en page : copie-la en une seule ligne ou utilise la version texte indiquée en fin de document.

9 Exemple Bash : calculer une note CVSS sur un vecteur inventé

Exemple avec des valeurs inventées, pas celles du labo : `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`
donne `6.1` et la qualification `moyenne`.

BASH

```
awk 'BEGIN {
    AV=0.85; AC=0.77; PR=0.85; UI=0.62; C=0.22; I=0.22; A=0.00;
    ISS = 1 - ((1-C) * (1-I) * (1-A));
    Impact = 7.52 * (ISS - 0.029) - 3.25 * ((ISS - 0.02)^15);
    Exploitability = 8.22 * AV * AC * PR * UI;
    Total = 1.08 * (Impact + Exploitability);
    if (Total > 10) Total = 10;
    scaled = int((Total * 100000) + 0.5);
    if (scaled % 10000 == 0) score = scaled / 100000; else score = (int(scaled / 10000) + 1) / 10;
    printf "score=%.1f\n", score;
}'
```

10 Exemple PowerShell 5.1 : même calcul sur un vecteur inventé

POWERSHELL

```
$AV = 0.85; $AC = 0.77; $PR = 0.85; $UI = 0.62; $C = 0.22; $I = 0.22; $A = 0.00
$ISS = 1 - ((1 - $C) * (1 - $I) * (1 - $A))
$Impact = 7.52 * ($ISS - 0.029) - 3.25 * [Math]::Pow($ISS - 0.02, 15)
$Exploitability = 8.22 * $AV * $AC * $PR * $UI
$Total = 1.08 * ($Impact + $Exploitability)
if ($Total -gt 10) { $Total = 10 }
$scaled = [int][Math]::Round($Total * 100000, 0, [MidpointRounding]::AwayFromZero)
if ($scaled % 10000 -eq 0) { $score = $scaled / 100000 } else { $score = ([Math]::Floor($scaled / 10000) + 1) / 10 }
'score={0:N1}' -f $score
```

11 Ce qu'il faut vérifier avant de répondre

- ◆ une seule ligne `faux_positif` et une seule ligne `preuve_insuffisante` doivent émerger du fichier de validation ;
- ◆ les identifiants `Vxx` n'ont aucune sévérité implicite ;
- ◆ tous les EPSS sont distincts ;
- ◆ un `kev=yes` faux positif sort du classement prioritaire tant que le faux positif est documenté ;
- ◆ la fiche actifs sert à départager les constats de même bande EPSS, pas à ignorer KEV.



Pingo veille sur ta progression

Retourne au laboratoire pour valider tes réponses. Bloqué ? Ouvre d'abord l'indice de la tâche, la correction vient ensuite. Pour copier les commandes sans coupure de ligne, ouvre la [version texte](#).